

Universidad de Murcia
Departamento de Informática

Práctica 4 – Programación en CUDA

Metodología de la Programación Paralela

José Luis García Valverde
jl.garciavalverde@um.es
Grupo 1.1

Eric Montalbán Atencia
eric.m.a@um.es
Grupo 1.3

Curso 2025/2026

Índice general

Contenido	1
Lista de figuras	2
1. Introducción	1
2. Desarrollo Experimental	2
2.1. Cuestión 1: Identificación de la GPU	2
2.1.1. Análisis de la herramienta <code>deviceQuery</code>	2
2.1.2. Resultados de ejecución	2
2.1.3. Resumen de características	3
2.2. Cuestión 2: Análisis de Ancho de Banda	3
2.2.1. Descripción del experimento	3
2.2.2. Resultados gráficos	4
2.2.3. Análisis cuantitativo	5
2.2.4. Discusión	5
2.3. Cuestión 3: Análisis del programa <code>cudaTemplate</code>	5
2.3.1. 3.a) Funcionalidad y lógica del kernel	5
2.3.2. 3.b) Gestión de memoria compartida	6
2.3.3. 3.c) Sincronización	6
2.3.4. 3.d) Soporte para bloques tridimensionales	6
3. Cuestión 4: Análisis de Rendimiento en Suma de Vectores	8
3.1. 4.a) Tiempos de ejecución y escalabilidad	8
3.2. 4.b) Implementación del Kernel	9
3.3. 4.c) Dimensionamiento del Grid	10
3.4. 4.d) Influencia del Warp en el Rendimiento	10

Índice de figuras

2.1. Resumen del análisis de rendimiento de memoria.	4
2.2. Detalle de ancho de banda por dirección de transferencia.	4

Capítulo 1

Introducción

La presente memoria documenta el desarrollo y los resultados obtenidos en la Práctica 4, cuyo eje central es la programación de alto rendimiento en dispositivos gráficos utilizando la arquitectura CUDA (*Compute Unified Device Architecture*).

El objetivo principal es comprender el modelo de programación heterogéneo CPU-GPU y analizar cómo las características del hardware influyen en el diseño del software. Para ello, el trabajo se estructura en cuatro bloques experimentales:

1. **Identificación del hardware:** Caracterización técnica de la GPU disponible mediante la API de Runtime de CUDA.
2. **Análisis de ancho de banda:** Estudio comparativo de las transferencias de memoria entre Host y Device, evaluando el impacto del uso de memoria paginada frente a memoria bloqueada (*pinned*).
3. **Programación de kernels (cudaTemplate):** Análisis de un kernel con gestión explícita de memoria compartida e índices tridimensionales.
4. **Optimización de operaciones vectoriales (vectorAdd):** Evaluación del impacto del tamaño de bloque y la ocupación de la GPU en el tiempo de ejecución.

A continuación, se detallan los procedimientos experimentales y las conclusiones técnicas derivadas de cada cuestión.

Capítulo 2

Desarrollo Experimental

2.1. Cuestión 1: Identificación de la GPU

2.1.1. Análisis de la herramienta deviceQuery

Para la caracterización del hardware se ha utilizado la utilidad `deviceQuery`, incluida en el SDK de CUDA. Este programa invoca la función `cudaGetDeviceProperties` para obtener una estructura de tipo `cudaDeviceProp`, la cual expone las especificaciones técnicas del dispositivo.

Los parámetros más relevantes analizados incluyen:

Capacidad de Cómputo: Determinada por el número de *Streaming Multiprocessors* (SM) y la cantidad de núcleos CUDA por SM.

Jerarquía de Memoria: Capacidad total de memoria global, tamaño de la memoria compartida por bloque y número de registros disponibles.

Restricciones de Ejecución: Límites máximos de hilos por bloque y dimensiones máximas para la organización de la rejilla (*grid*) y los bloques.

2.1.2. Resultados de ejecución

La ejecución en el nodo de pruebas confirmó la detección de una tarjeta gráfica NVIDIA GeForce GT 1030, arrojando la siguiente salida:

```
1 ./deviceQuery Starting...
2
3  CUDA Device Query (Runtime API) version (CUDA static linking)
4
5  Detected 1 CUDA Capable device(s)
6
7  Device 0: "NVIDIA GeForce GT 1030"
8      CUDA Driver Version / Runtime Version      12.2 / 12.5
```

```

9      CUDA Capability Major/Minor version number:    6.1
10     Total amount of global memory:                  2000 MBytes (2097020928
bytes)
11     ( 3) Multiprocessors, (128) CUDA Cores/MP:      384 CUDA Cores
12     GPU Max Clock rate:                             1430 MHz (1.43 GHz)
13     Memory Clock rate:                             1050 Mhz
14     Memory Bus Width:                              64-bit
15     L2 Cache Size:                                  524288 bytes
16     Total amount of shared memory per block:        49152 bytes
17     Total number of registers available per block:  65536
18     Warp size:                                       32
19     Maximum number of threads per multiprocessor:   2048
20     Maximum number of threads per block:            1024
21     Max dimension size of a thread block (x,y,z):  (1024, 1024, 64)
22     Max dimension size of a grid size    (x,y,z):  (2147483647, 65535, 65535)
23
24     Result = PASS

```

Listing 2.1: Salida de ejecución de deviceQuery

2.1.3. Resumen de características

A partir de la salida anterior, se han sintetizado las especificaciones técnicas clave en la Tabla 2.1.

Tabla 2.1: Especificaciones técnicas de la NVIDIA GeForce GT 1030

Parámetro	Valor
Número de Streaming Multiprocessors (SM)	3
CUDA Cores por SM	128
Total de CUDA Cores	384
Máx. Threads por SM	2048
Máx. Threads por Bloque	1024
Memoria Global Total	2000 MB
Registros por Bloque	65536
Memoria Compartida por Bloque	49152 bytes (48 KB)
Dimensiones Máx. del Grid	(2147483647, 65535, 65535)
Dimensiones Máx. del Bloque	(1024, 1024, 64)

2.2. Cuestión 2: Análisis de Ancho de Banda

2.2.1. Descripción del experimento

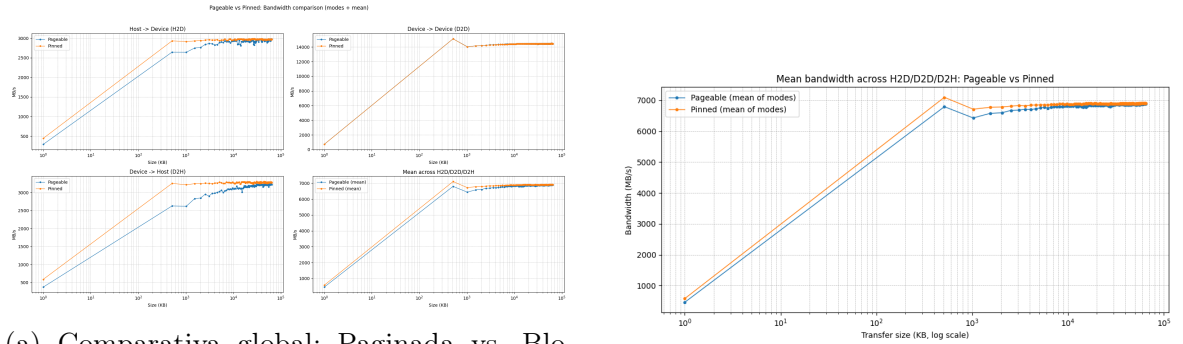
Se ha evaluado el rendimiento de las transferencias de memoria utilizando la utilidad `bandwidthTest`. El experimento compara la eficiencia de la memoria paginada

(*pageable*) frente a la memoria bloqueada (*pinned* o *page-locked*) en tres direcciones de flujo de datos:

- **H2D:** Host a Device.
- **D2D:** Device a Device (copia interna).
- **D2H:** Device a Host.

2.2.2. Resultados gráficos

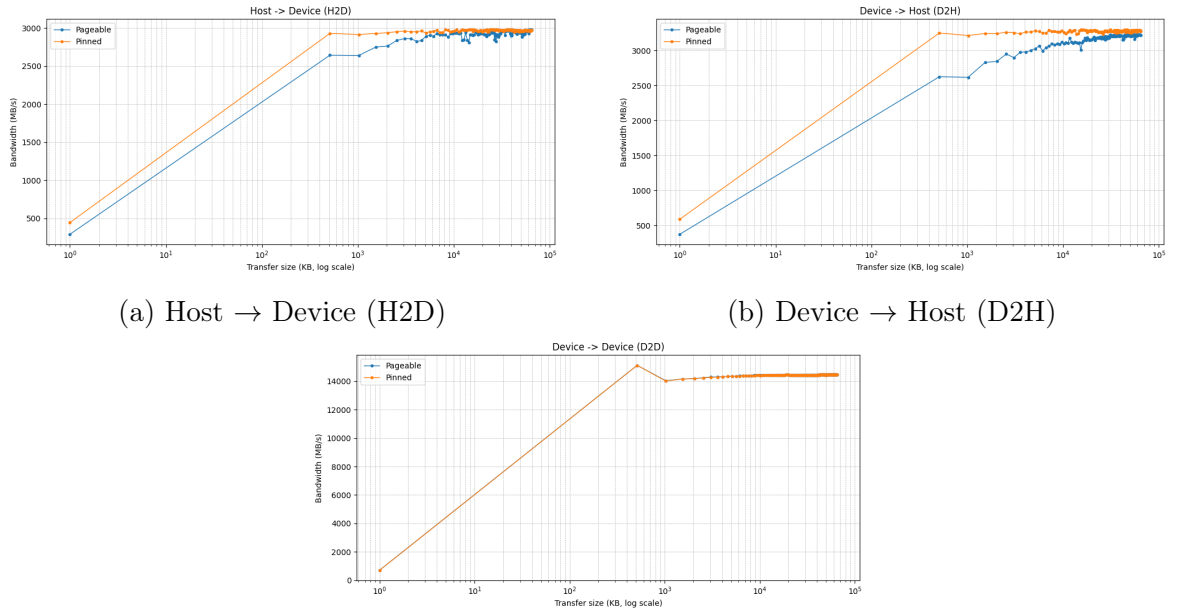
Las Figuras 2.1 y 2.2 ilustran la evolución del ancho de banda en función del tamaño de la transferencia.



(a) Comparativa global: Paginada vs. Bloqueada.

(b) Medias globales por tipo de transferencia.

Figura 2.1: Resumen del análisis de rendimiento de memoria.



(a) Host → Device (H2D)

(b) Device → Host (D2H)

(c) Device → Device (D2D)

Figura 2.2: Detalle de ancho de banda por dirección de transferencia.

2.2.3. Análisis cuantitativo

La Tabla 2.2 muestra los valores medios de ancho de banda obtenidos.

Tabla 2.2: Comparativa de ancho de banda medio (MB/s)

Tipo	Pageable	Pinned	Ganancia (Speedup)
H2D (Host a Device)	2901.34	2945.55	1.015×
D2D (Device a Device)	14324.27	14322.58	1.000×
D2H (Device a Host)	3136.82	3254.65	1.038×

2.2.4. Discusión

Los resultados corroboran las ventajas arquitectónicas del uso de memoria *pinned*:

- **Transferencias Host-Device:** Se observa una mejora en H2D y D2H. Al usar memoria bloqueada, el sistema operativo garantiza que las páginas residen en la memoria física (no se intercambian a disco), permitiendo que el motor DMA de la GPU acceda directamente a la memoria del Host. Esto elimina la necesidad de que la CPU copie los datos a un buffer intermedio del driver, reduciendo latencia y carga de CPU.
- **Device-Device:** Como era esperado, el tipo de memoria en el Host es irrelevante para copias internas en la VRAM de la GPU, obteniéndose resultados idénticos.
- **Asimetría:** Existe un rendimiento ligeramente superior en lecturas (D2H) frente a escrituras (H2D), con una ganancia máxima del 3.8% al usar memoria *pinned* en sentido D2H.

2.3. Cuestión 3: Análisis del programa cudaTemplate

2.3.1. 3.a) Funcionalidad y lógica del kernel

El kernel `foo` implementa un patrón de acceso diseñado para ilustrar la gestión manual de la jerarquía de memoria. Las operaciones por hilo son:

1. **Cálculo de índices (Linealización):** Se transforman las coordenadas del hilo a índices lineales. Primero se obtiene el índice local dentro del bloque (tid_b) y posteriormente el índice global (tid_g), considerando la geometría de la rejilla.
2. **Carga a memoria compartida:** Cada hilo transfiere un elemento desde la memoria global (`gid_d`) hacia su posición en la memoria compartida (`shared_mem`).
3. **Cómputo local:** Se actualiza el valor en memoria compartida sumando el índice global y un valor recuperado de la memoria constante (`const_d`).

4. **Escritura de resultados:** El valor procesado se escribe de vuelta en la memoria global.

2.3.2. 3.b) Gestión de memoria compartida

La memoria compartida se asigna dinámicamente al lanzar el kernel. El tamaño necesario es:

$$\text{shared_mem_size} = \text{block.x} \times \text{block.y} \times \text{block.z} \times \text{sizeof(int)}$$

Para garantizar que esta solicitud no exceda los 49152 bytes disponibles por bloque en la GT 1030, se proponen dos validaciones:

Validación Estática

Se actualizó la constante en `cudaTemplate.cu`: `#define SH_MEM_SIZE (48 * 1024)`. Esto ajusta la aserción a los límites reales de la arquitectura Pascal.

Validación Dinámica (Recomendada)

Para mayor portabilidad, se puede consultar el límite en tiempo de ejecución:

```
1 /*
2 cudaDeviceProp deviceProp;
3 checkCudaErrors(cudaGetDeviceProperties(&deviceProp, 0));
4 if (shared_mem_size > deviceProp.sharedMemPerBlock) {
5     fprintf(stderr, "Error: Memoria compartida excede el limite (%zu)\n",
6             deviceProp.sharedMemPerBlock);
7     exit(EXIT_FAILURE);
8 }
9 */
```

Listing 2.2: Validación dinámica de recursos

2.3.3. 3.c) Sincronización

En este diseño, existe una correspondencia uno a uno entre hilos y posiciones de memoria compartida. Dado que ningún hilo lee datos escritos por otro hilo (ausencia de dependencias cruzadas), no hay riesgo de condiciones de carrera. Por tanto, las barreras `__syncthreads()` son redundantes y podrían eliminarse para optimizar el rendimiento.

2.3.4. 3.d) Soporte para bloques tridimensionales

Se refactorizó el código para soportar coordenadas 3D. Las fórmulas clave implementadas para la dimensión z son:

- **Tamaño del bloque:** $blockSize = D_x \cdot D_y \cdot D_z$
- **Índice local (tid_b):** $tid_b = threadIdx.z \cdot (D_y \cdot D_x) + threadIdx.y \cdot D_x + threadIdx.x$
- **ID de bloque ($blockId$):** $blockId = blockIdx.z \cdot (G_x \cdot G_y) + blockIdx.y \cdot G_x + blockIdx.x$

Esto habilita ejecuciones volumétricas como:

```
1 ./cudaTemplate --gsx=4 --gsy=2 --gsz=2 --bsx=8 --bsy=8 --bsz=1
```

Capítulo 3

Cuestión 4: Análisis de Rendimiento en Suma de Vectores

En esta sección se analiza la ejecución del programa `vectorAdd`, variando el número de elementos del vector y la configuración de hilos por bloque (`tpb`). Los datos provienen de los archivos de log ubicados en `Codigo/vectorAdd`.

3.1. 4.a) Tiempos de ejecución y escalabilidad

La Tabla 3.1 resume los tiempos medios y la desviación estándar tras 5 repeticiones por configuración.

Tabla 3.1: Tiempos de ejecución (ms) para distintas configuraciones de `nelem` y `tpb`

Configuración (Elementos / TPB)	Media (ms)	Std (ms)
<i>Tamaño pequeño: 1.000.000 elementos</i>		
1M / 128	5.5336	0.0603
1M / 256	5.5377	0.0174
1M / 1024	5.5785	0.0641
1M / 32	6.0064	1.0581
1M / 512	6.7275	2.5834
<i>Tamaño medio: 10.000.000 elementos</i>		
10M / 128	49.5668	0.3714
10M / 256	49.7347	0.2984
10M / 512	49.8106	0.4867
10M / 1024	51.7977	4.3757
10M / 32	55.0660	11.3435
<i>Tamaño grande: 50.000.000 elementos</i>		
50M / 1024	244.1822	0.6611
50M / 256	244.2601	0.3637
50M / 32	245.3972	0.6971

Análisis de resultados

- **Configuraciones óptimas:** Para cargas medias (10M), los tamaños de bloque intermedios (128 y 256 hilos) ofrecen el mejor rendimiento y estabilidad.
- **Penalización por bloques pequeños:** El uso de `tpb=32` resulta consistentemente más lento y presenta una desviación estándar alta. Esto se debe a una baja ocupación del hardware y un mayor overhead en la gestión de un número excesivo de bloques pequeños.
- **Saturación en bloques grandes:** Para 50M de elementos, las diferencias entre 256 y 1024 hilos se diluyen, indicando que el cuello de botella se desplaza hacia el ancho de banda de memoria global más que hacia la capacidad de cómputo.

3.2. 4.b) Implementación del Kernel

El código del kernel (`vectorAdd_kernel.cu`):

```

1 __global__ void vectorAdd(const float *X, const float *Y, float *V, int
  numElements)
2 {
3     int tid = blockDim.x * blockIdx.x + threadIdx.x;
4     if (tid < numElements) {
5         V[tid] = X[tid] + Y[tid];
6     }

```

Listing 3.1: Kernel `vectorAdd`

En el, cada hilo calcula un identificador único global (`tid`). La condición `if (tid < numElements)` es crítica para evitar accesos ilegales a memoria cuando el número total de hilos lanzados excede el tamaño del vector.

3.3. 4.c) Dimensionamiento del Grid

El número de bloques necesarios se calcula en el Host mediante la fórmula de redondeo hacia arriba:

$$blocksPerGrid = \left\lceil \frac{numElements}{threadsPerBlock} \right\rceil = \frac{numElements + threadsPerBlock - 1}{threadsPerBlock} \quad (3.1)$$

Esta estrategia asegura que se generen suficientes hilos para cubrir todos los elementos del vector. Cuando *numElements* no es múltiplo de *threadsPerBlock*, el último bloque contendrá hilos "inactivos" (cuyo *tid* $\geq numElements$), los cuales serán descartados por el condicional dentro del kernel, garantizando la corrección del programa.

3.4. 4.d) Influencia del Warp en el Rendimiento

El **warp** es la unidad fundamental de despacho de instrucciones en la arquitectura CUDA, constituida por 32 hilos que se ejecutan en modo SIMT (*Single Instruction, Multiple Threads*). La alineación con el tamaño del warp es crucial por tres motivos:

1. **Subutilización (Underutilization):** Si el tamaño del bloque no es múltiplo de 32, el último warp del bloque estará parcialmente vacío. Los hilos inexistentes ocupan "slots" de ejecución sin realizar trabajo útil, desperdiciando ciclos de computación.
2. **Divergencia de ramas:** Aunque no aplica en este kernel simple, en algoritmos complejos, si los hilos de un mismo warp toman caminos distintos en un condicional, la ejecución se serializa, degradando el rendimiento.
3. **Coalescencia de memoria:** El hardware de memoria está optimizado para servir peticiones de 32 hilos consecutivos simultáneamente. Si los bloques o los accesos no están alineados, se requieren más transacciones de memoria para servir la misma cantidad de datos.

Los resultados empíricos (Tabla 3.1) donde `tpb=32` muestra peor rendimiento y alta variabilidad, confirman que tamaños de bloque muy reducidos impiden ocultar eficazmente la latencia de memoria y aumentan la sobrecarga de planificación de warps.